Openi nCol ab

In [ ]:

```
import numpy as np
#print(np.__version__)
```

In [ ]:

```
class Layer:
    def __init__(self):
        self.input = None
        self.output = None

    def forward(self, input):
        pass

    def backward(self,output_gradient, learning_rate):
        pass
```

In [ ]:

```
weight_list = []
bias_list = []
```

In [ ]:

```
class Dense(Layer):
    def __init__(self, input_size, output_size):
        self.weights = np.random.randn(output_size, input_size)
        self.bias = np.random.randn(output_size, 1)

    def forward(self, input):
        self.input = input
        return np.dot(self.weights, self.input) + self.bias

    def backward(self, output_gradient, input_gradient):
        weights_gradient  =  np.dot(output_gradient,  self.input.T)
        input_gradient = np.dot(self.weights.T, output_gradient)

        self.weights -= learning_rate * weights_gradient
        self.bias -= learning_rate * output_gradient

        weight_list.append(self.weights)
        bias_list.append(self.bias)

        return input_gradient
```

In [ ]:

```
def mse(y_true, y_pred):
    return np.mean(np.power(y_true - y_pred, 2))

def mse_prime(y_true, y_pred):
    return 2 * (y_pred - y_true) / np.size(y_true)
```

In [ ]:

```
class Activation(Layer):
    def __init__(self, activation, activation_prime):
        self.activation = activation
        self.activation_prime = activation_prime

    def forward(self, input):
        self.input = input
        return self.activation(self.input)
```

```
    def backward(self, output_gradient, learning_rate):
        return np.multiply(output_gradient, self.activation_prime(self.inp
```

In [ ]:

```
class Linear(Activation):
    def __init__(self):
        def linear(x):
            return x

        def linear_prime(x):
            return 1

        super().__init__(linear, linear_prime)
```

In [ ]:

```
X = np.array([0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7])
Y = np.array([1.2, 1.4, 1.55, 1.75, 2.01, 2.2, 2.35])

network = [
    Dense(1, 1),
    Linear()
]

epochs = 100
learning_rate = 0.5
errorlist = []

for e in range(epochs):
    error = 0
    for x,y in zip(X,Y):
        output = x
        for layer in network:
            output=layer.forward(output)
        error += mse(y,output)
        grad = mse_prime(y,output)

        for layer in reversed(network):
            grad = layer.backward(grad, learning_rate)

    print(f"{e + 1}/({epochs}, error={error}")
    errorlist.append(error)
```

```
1/(100,        error=1.1434355931608384
2/(100,       error=0.5333515963938432
3/(100,     error=0.38729568220421745
4/(100,     error=0.28233655204173663
5/(100,       error=0.2068396301161926
6/(100,     error=0.15247554016694018
7/(100,     error=0.11327904365778226
8/(100,    error=0.08497657272596489
9/(100,      error=0.0645052682395757
10/(100,    error=0.04966895125831257
11/(100,  error=0.038892012462327784
12/(100,   error=0.031043325061376272
13/(100,   error=0.025310237026661388
14/(100,    error=0.021108382262761134
15/(100,     error=0.018017111382004976
16/(100,   error=0.015733267418923887
17/(100,
error=0.014038040663364643
18/(100, error=0.012773258442994779
19/(100,   error=0.011824356925230732
20/(100, error=0.011108177868117136
```

21/(100,
error=0.010564207987488593
22/(100,
error=0.010148286446965797
23/(100,
error=0.009828081285469652
24/(100, error=0.009579834267811924
25/(100,
error=0.009386015769682855
26/(100, error=0.009233632997211402
27/(100, error=0.009113007610374587
28/(100,
error=0.009016890903868294
29/(100,
error=0.008939821987030062
30/(100, error=0.008877661106842152
31/(100, error=0.008827249386251098
32/(100, error=0.00878615995773198
33/(100, error=0.008752515299699786
34/(100,
error=0.008724852633013855 35/(100,
error=0.008702024294878652
36/(100, error=0.008683123642072183
37/(100,
error=0.008667429648422867
38/(100,
error=0.008654365241857524
39/(100, error=0.00864346578112683
40/(100, error=0.00863435504973118
41/(100,
error=0.008626726850858519
42/(100,
error=0.008620330798434267
43/(100, error=0.008614961270302327
44/(100,
error=0.008610448759288814
45/(100, error=0.0086066530545969
46/(100, error=0.00860345782986687
47/(100,
error=0.008600766319863909
48/(100, error=0.00859849784561232
49/(100,
error=0.008596585005427515
50/(100, error=0.008594971392178418
51/(100, error=0.008593609729178506
52/(100,
error=0.008592460341228259
53/(100,
error=0.008591489895588335
54/(100,
error=0.008590670361571028 55/(100,
error=0.008589978148106721 56/(100,
error=0.008589393386875168
57/(100,
error=0.008588899335003236
58/(100,
error=0.008588481876346506
59/(100, error=0.008588129104326213
60/(100,
error=0.008587830972442041
61/(100, error=0.008587579001084157
62/(100,
error=0.008587366031292005
63/(100,
error=0.008587186017737447
64/(100,
error=0.008587033854535027
65/(100, error=0.00858690522856973

```
85/(100,    error=0.00858622635109741
86/(100,  error=0.008586222566842673
87/(100,   error=0.008586219367378712
88/(100,  error=0.008586216662334292
89/(100,  error=0.008586214375304492
90/(100,   error=0.008586212441691785
91/(100,    error=0.00858621080688167
92/(100,   error=0.00858620942469942
93/(100,  error=0.008586208256105936
94/(100,  error=0.008586207268095168
95/(100,  error=0.008586206432761418
96/(100,   error=0.008586205726511418
97/(100,    error=0.00858620512939791
98/(100, error=0.008586204624555938
99/(100,  error=0.008586204197726835
100/(100,
error=0.008586203836855188
```

In [ ]:
```
X1 = [0.15]

for x in X1:
    output = x
    for layer in network:
        output=layer.forward(output)
    print(output)
```

```
[[1.31024375]]
```

In [ ]:
```
import matplotlib.pyplot as plt
```

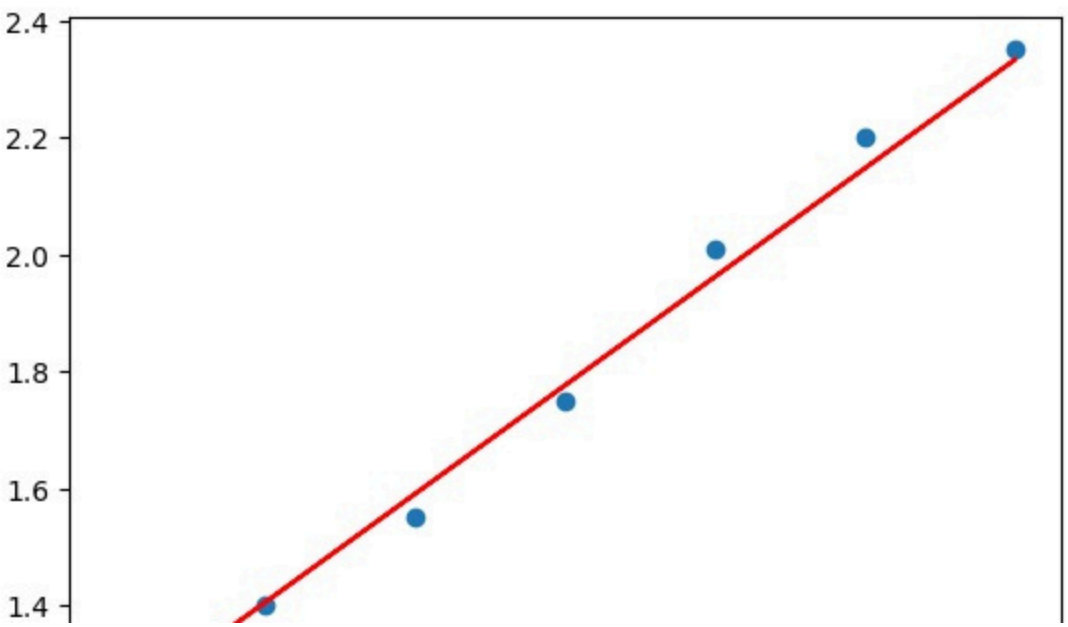
In [ ]:
```
w = weight_list[100][0][0]
```

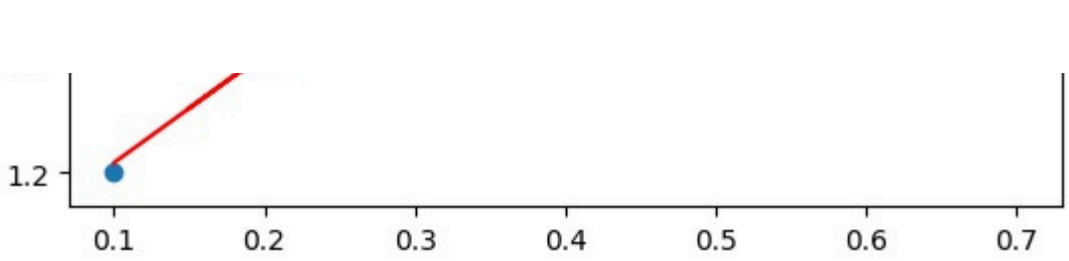
In [ ]:
```
b = bias_list[100][0][0]
```

In [ ]:
```
plt.scatter(X,Y)

x1 = [0.1,0.2,0.3,0.4,0.5,0.6,0.7,0.15]
y1 = [(w*i + b) for i in x1]
plt.plot(x1, y1, color='r')
```

Out[ ]:
```
[<matplotlib.lines.Line2D at 0x7958b533b890>]
```




1.2
0.1
0.2
0.3
0.4
0.5
0.6
0.7